Lab 3 : Formulaires & Communication (Lifting State Up)

Cours Développement Web Moderne

1 Introduction

Dans React, le flux de données est unidirectionnel (du parent vers l'enfant). Ce laboratoire explore comment :

- Créer des **composants contrôlés** (lier un input à un état).
- Pratiquer le **"Lifting State Up"** : faire remonter une information d'un composant enfant vers son parent via des fonctions de rappel (*callbacks*) passées en *props*.

2 Concepts Clés

2.1 Inputs Contrôlés

Un input est dit "contrôlé" lorsque sa valeur est pilotée par le `state` de React.

```
const [value, setValue] = useState("");
// ...
<input value={value} onChange={(e) => setValue(e.target.value)} />
```

2.2 Lifting State Up

Pour qu'un enfant modifie l'état de son parent, le parent doit lui passer une fonction en tant que *prop*. L'enfant appelle cette fonction pour envoyer des données vers le haut.

3 Boilerplate : Structure de départ

3.1 Composant `TaskInput.jsx`

Ce composant gère son propre état interne pour le champ de texte.

```
import { useState } from 'react';

const TaskInput = ({ onAddTask }) => {
  const [inputValue, setInputValue] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    if (inputValue.trim()) {
      onAddTask(inputValue); // On fait remonter l'info
      setInputValue(""); // On vide le champ
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
```

```
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
        placeholder="Ajouter une tache..."
      />
      <button type="submit">Ajouter</button>
    </form>
  );
};

export default TaskInput;
```

3.2 Le Parent App.jsx

```
import { useState } from 'react';
import TaskInput from './components/TaskInput';

function App() {
  const [tasks, setTasks] = useState([]);

  const addTask = (text) => {
    // Logique pour ajouter une tache ici
  };

  return (
    <div className="container">
      <h1>Ma Todo List</h1>
      <TaskInput onAddTask={addTask} />
      <ul>{/* La liste des taches viendra ici */}</ul>
    </div>
  );
}
```

4 Style suggéré (CSS)

```
/* App.css */
.container { max-width: 400px; margin: 2rem auto; font-family: sans-
    serif; }
.task-item {
  display: flex;
  justify-content: space-between;
  padding: 8px;
  border-bottom: 1px solid #eee;
}
.delete-btn { color: red; cursor: pointer; border: none; background:
    none; }
```

5 Le Challenge

Challenge : Todoist Core

Votre objectif est de rendre la liste fonctionnelle et interactive.

1. **Ajout dynamique** : Complétez la fonction `addTask` pour ajouter un nouvel objet tâche (avec un `id` unique et un `text`) au tableau `tasks`.
2. **Affichage** : Utilisez `.map()` pour afficher la liste des tâches dans `App.jsx`.
3. **Suppression** : Implémentez une fonction `deleteTask(id)` dans `App.jsx` qui filtre le tableau pour retirer la tâche sélectionnée.
4. **Communication descendante** : Passez la fonction de suppression à chaque élément de liste et déclenchez-la au clic sur un bouton "Supprimer" ou une icône.